

Suivi du projet

Base de donnée

Sauvegarde d'équipe

Afin de pouvoir sauvegarder et charger des équipes, nous avons ajouté une nouvelle table `teams`, qui contient l'identifiant du joueur associé à l'équipe ainsi que le nom de l'équipe.

Nous avons également ajouté une table `team_members`, liée à la table `teams`, qui contient les identifiants des Bugémons appartenant à chaque équipe.

Sauvegarde de progression

Pour sauvegarder la progression dans la Tour du NO, nous avons ajouté une table `tower_saves` qui contient l'identifiant du joueur associé à l'étage actuel du joueur, les salles complétées dans cet étage, ainsi que l'identifiant de l'équipe choisie pour la tour. Cette équipe est différenciée de celles sauvegardées lors de la création d'équipe grâce à un booléen `tower_team` dans la table `teams`. Cela permet d'éviter de charger l'équipe utilisée dans la tour du NO quand le joueur veut choisir une de ses équipes sauvegardées.

Social

Leaderboard

Nous avons ajouté un attribut "points" à chaque utilisateur dans la base de données. Ce nombre de points augmente après chaque victoire.

Pour l'affichage, les points sont visibles dans le Social Panel, aux côtés du nom du joueur et de son score total.

Combats multijoueur

Nous avons ajouté un système de requêtes pour défier ses amis. Celui-ci est identique au système de demandes d'amis.

Une nouvelle classe `MultiBattleSession` a été créée dans la couche des modèles. Celle-ci contient les informations relatives aux combats multijoueurs en cours dans le serveur. Un repository et un service associés ont été créés. Le repository n'utilise pas la base de données mais enregistre plutôt les sessions dans une `HashMap`, car celles-ci ne sont pas considérées comme données persistantes.

Lorsqu'une requête de combat est acceptée, les deux joueurs sont amenés à choisir leur équipe. Le combat commence une fois les deux équipes choisies. Une vue `WaitWindow` fait patienter un joueur lorsqu'il doit attendre l'autre entre deux vues.

Les combats multijoueurs dans les contrôleurs sont différenciés des combats singleplayer par la présence d'un attribut Opponent (un PlayerDTO mis à null pour les combats solo).

Censure/adoucissement des messages inappropriés

Pour l'histoire liée au filtrage du chat, nous avons ajouté une classe InappropriateWordFilter dans la couche service. Avant l'enregistrement d'un message, ChatService applique ce filtre au contenu envoyé, puis stocke uniquement la version adoucie du message. Le filtrage est volontairement centralisé côté service afin d'éviter de dupliquer la logique dans les contrôleurs ou dans le repository.

Le filtre détecte les mots composés de lettres Unicode, ce qui permet de gérer les accents et de conserver la ponctuation autour des mots. Les comparaisons sont insensibles à la casse et aux accents grâce à une normalisation préalable : par exemple, « IMBÉCILE » et « imbécile » sont traités comme le même mot. Lorsqu'un mot est reconnu comme inapproprié, seule sa partie centrale est remplacée par des astérisques ; la première et la dernière lettre restent visibles, conformément à la demande client. Des tests unitaires vérifient notamment le masquage, la conservation de la ponctuation, la gestion des accents et l'intégration du filtre dans ChatService.

Communications client-serveur

Messages

Nous utilisons des objets sérialisables que nous appelons Messages comme base pour les communications entre serveur et clients. Ce système était à l'origine pensé de façon symétrique. Cependant, nous les avons utilisés en pratique comme un système asymétrique de requêtes client – réponse serveur. Nous les avons donc renommés Request et Response pour refléter cet usage et afin d'un peu mieux généraliser leur usage.

Gestion d'erreurs

Nous avons amélioré la gestion d'erreurs du projet en remplaçant progressivement les erreurs génériques par des exceptions dédiées dans le package `ulb.exceptions`. L'objectif est de rendre les erreurs plus explicites, plus faciles à comprendre et plus simples à traiter selon leur origine : communication réseau, accès aux données, chargement de vues JavaFX, authentification, logique de jeu, etc.

Exception	Usage / comportement
-----------	----------------------

CommunicationException	Utilisée lorsqu'un problème survient dans la communication client-serveur, par exemple lors de l'ouverture d'une connexion, de l'envoi ou de la réception d'une requête/réponse réseau.
DataAccessException	Exception générale pour les erreurs techniques liées aux données persistantes. Elle sert de base pour regrouper les problèmes d'accès ou de transformation des données.
LoadException	Spécialisation de DataAccessException, utilisée lorsqu'un chargement échoue, par exemple lors de la lecture de données de jeu ou de données venant de la base de données.
MappingException	Spécialisation de DataAccessException, utilisée lorsqu'un DTO ne peut pas être converti correctement en objet du modèle. Elle permet d'identifier les erreurs de transformation entre les couches.
DuplicateElementException	Utilisée lorsqu'on tente d'insérer un élément déjà présent, par exemple une capacité, une espèce de Bugémon ou un objet déjà existant dans la base de données.
EntityNotFoundException	Utilisée lorsqu'une entité demandée n'existe pas, par exemple lorsqu'un identifiant ne correspond à aucun élément connu.
InvalidCredentialsException	Utilisée lors de la connexion si les identifiants fournis ne sont pas valides,

	notamment en cas de mauvais mot de passe.
UserAlreadyExistsException	Utilisée lors de l'inscription si le nom d'utilisateur existe déjà dans la base de données.
GameException	Exception générale pour les erreurs bloquantes liées à la logique du jeu. Elle sert de base pour les erreurs métier côté jeu.
UserFacingException	Spécialisation de GameException, utilisée pour les erreurs qui peuvent être affichées directement au joueur, par exemple lorsqu'une action demandée est impossible dans l'état courant du jeu.
ViewLoadException	Utilisée lorsqu'une vue JavaFX ou un fichier FXML ne peut pas être chargé. Elle permet d'éviter de laisser remonter directement les exceptions techniques de JavaFX.

En plus de ces exceptions spécialisées, nous avons remplacé les affichages directs d'erreurs par l'utilisation de `java.util.logging.Logger`. Les logs permettent d'indiquer les erreurs potentielles avec un niveau adapté (`WARNING`, `SEVERE`, etc.), mais aussi de signaler les moments où l'application fonctionne correctement (`INFO`), par exemple lorsque le serveur démarre avec un message comme `Server is running ...`

Toutes les occurrences de `"System.out.println(...)"`, `"printStackTrace"`, ... ont été enlevées.